

Reto técnico: SSr Machine Learning Engineer

Te damos la bienvenida al proceso de selección. Este reto consiste en diseñar y prototipar un sistema robusto de **Moderación de Imágenes** para garantizar la calidad del contenido y el cumplimiento de las políticas comerciales del marketplace.

Dispones de un plazo máximo de una semana para entregar tu solución, el cual comienza a contar desde la recepción de este documento.

Contexto

Actualmente, el equipo de Moderaciones utiliza un flujo asíncrono basado en OCR + Regex para identificar textos y dar de baja imágenes que contengan promociones no permitidas (por ejemplo, "50% off"), promesas de entrega (como "¡Entrega el mismo día!") o textos superpuestos de campañas específicas (por ejemplo, logotipos de Black Friday o Hot Sale). Este proceso se activa mediante una cola de eventos generada por la creación de ítems o la modificación de imágenes asociadas a ítems existentes, lo que representa un tráfico promedio de 60k RPM con picos de hasta 100k RPM.

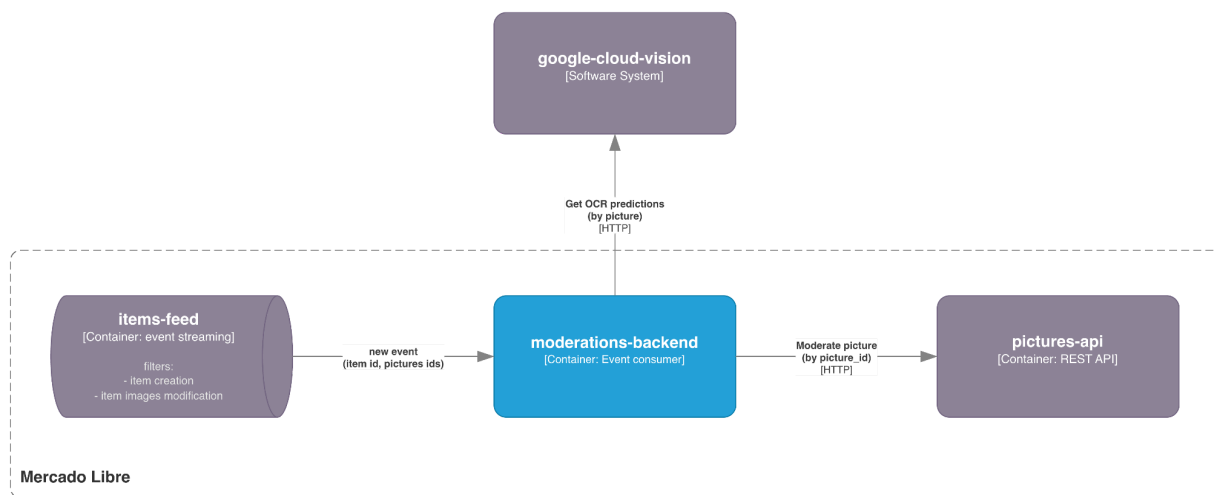


Fig. 1. Arquitectura simplificada para la moderación de imágenes

Esta iniciativa surge ante la necesidad urgente de mitigar la alta tasa de falsos positivos (aún no cuantificada) del sistema actual, la cual impacta de forma negativa en la calidad del contenido del marketplace y en la experiencia del usuario. La solución vigente carece de

robustez frente a la diversidad de imágenes y textos superpuestos, lo que motiva la búsqueda de un enfoque técnico más avanzado y preciso.

Objetivo

Diseñar e implementar una prueba de concepto (POC) de un sistema de detección de imágenes que infrinja la política en cuestión, con el objetivo de mejorar la precisión sin sacrificar la cobertura del proceso.

Requerimientos del Sistema

Requerimientos Funcionales

- El sistema debe alcanzar una precisión $\geq 95\%$ y un recall $\geq 95\%$. Asimismo, la propuesta debe incluir el diseño del golden set que servirá como criterio de aceptación, especificando su composición, tamaño, distribución de clases y el proceso de etiquetado.
- El sistema debe exponer una función en Python que reciba la URL de una imagen (picture_url) y devuelva un diagnóstico con al menos dos campos: has_infraction (booleano) y evidence (string).

Requerimientos No Funcionales

- **Latencia:** El percentil 99 (p99) del tiempo de respuesta por imagen no debe superar los 5000 ms.
- **Interpretabilidad:** El sistema debe ser capaz de justificar cada clasificación de manera clara y comprensible. El campo evidence devuelto debe identificar los elementos específicos de la imagen que motivaron dicho diagnóstico.
- **Optimización de Costos:** Las soluciones propuestas deben minimizar el uso de APIs externas de OCR. Se priorizarán aquellos enfoques que eviten dependencias de terceros en el camino crítico de la inferencia.
- **Monitoreo:** La propuesta debe describir una estrategia de monitoreo detallada y especificar qué señales o métricas dispararán una alerta o una revisión del modelo.
- **Iteración:** La propuesta debe contemplar una estrategia de iteración que describa: (1) cómo se capturan y priorizan los falsos positivos y negativos en producción, (2) el mecanismo de actualización del modelo ante nuevos patrones, y (3) cómo se valida una nueva versión antes de reemplazar la que se encuentra en producción. No se requiere implementación en esta etapa.
- **Extensibilidad y Agilidad:** El diseño arquitectónico debe permitir la incorporación de nuevas etiquetas o criterios de moderación sin requerir cambios estructurales en el

pipeline de inferencia. La propuesta debe precisar qué componentes se modificarían y cuáles permanecerían estables ante este tipo de evolución.

- **Restricciones de Infraestructura:** El diseño debe considerar entornos de producción basados en instancias con GPU de 16 GB o 24 GB de VRAM, o instancias con CPU de hasta 4 cores y entre 2 y 32 GB de RAM. Asimismo, la solución debe estar diseñada para permitir la ejecución de múltiples workers por instancia.

Instrucciones por Perfil

1. Revisa el Anexo 1 que contiene la información necesaria para obtener los datos requeridos para resolver la prueba.
2. Comprende el problema utilizando los datos proporcionados.
3. Diseña tu solución, considerando tanto la POC como el escenario ideal definitivo (end game).
4. Desarrolla una prueba de concepto de un sistema en Python que permita identificar imágenes con infracciones.
5. Genera un reporte detallado en el que expliques la metodología aplicada, los resultados obtenidos y los puntos que quedaron pendientes.

Entregables

Se debe entregar:

- **Componente Escrito:** Presentación o documento que contenga la documentación técnica de la solución, diagramas, proceso de diseño y de experimentación, además de una sección explicativa sobre cómo y con qué propósito se utilizaron agentes de IA en el desarrollo de la prueba.
- **Componente de Código:** Código fuente de la POC en GitHub. Puede presentarse como un Jupyter Notebook o scripts en Python.